Matija Jakovac, UPC

Report on CPS Project (Technology - Constraint Programming)

1. Overview

This report describes the variables, constraints, and overall modeling strategy used in the implementation of a solution to the Minimum Consistent Finite Automata problem using **constraint programming** in **Gecode**. It attempts to find the smallest deterministic finite automaton (DFA) consistent with a given sample of accepted and rejected binary sequences.

2. Variables

The model defines the following key variables:

- ***BoolVarArray accepted***: A boolean variable array of size n (number of states), where accepted[i] is 1 if state i is an accepting state.
- ***IntVarArray reading_zero***: An integer variable array of size n where reading_zero[i] gives the destination state when reading a 0 from state i.
- ***IntVarArray reading_one***: An integer variable array of size n where reading_one[i] gives the destination state when reading a 1 from state i.
- ***IntVarArray f***: Created locally in the simulation of each sequence, representing the sequence of states visited when processing that input (accept or reject)

3. Functions

The program has two key functions:

- *void simulate_sequence(const vector<bool>& bits, int no_states, int expected_accept)* – simulates one given sequence (acceptable or rejectable) and make the necessary constraint for given sequence based on which bit is next and which is current state

- *vector<vector<bool>> read_sequences(istream& in)* – reads the sequence of *n* bits and makes a *vector<bool>* out of it, out of all sequences also makes and returns a *vector<vector<bool>>*

4. Constraints

The following constraints are enforced in the model:

- The initial **state** for all simulations is fixed as **state 0 – 'rel'** constraint.
- For each bit in a sequence, the appropriate transition (0 or 1) is followed based on the **current** state – **'element'** constraint.
- At the end of a sequence, the state reached must be accepting if the sequence is in the

accepted list, and rejecting if in the **rejected** list – **'rel'** constraint.
- The final state's acceptance is enforced by linking it to the accepted variable using an **'element'** constraint.
- Branching strategies used: **INT_VAR_SIZE_MIN** and **INT_VAL_MIN** for reading_zero and reading_one; **BOOL_VAR_NONE** and **BOOL_VAL_MIN** for accepted.

5. Search Strategy

The model uses a depth-first search (**DFS**) to explore possible DFA configurations. The number of states *n* is increased starting from 1 until a consistent DFA is **found**.

6. Conclusion

- The model is designed to find the minimal number of states needed for a consistent DFA.
- It outputs the DFA as a list of transitions and acceptance flags for each state as well as provided input for corresponding output.